

Mapping Reads

Purpose of Read Mapping

- Identify the origin of a read
- RNA-seq: Infer the gene that was expressed and that generated the read
- BUT: Is there always a unique origin for a read?

Sequence Alignment

- Optimization problem: Find the alignment with the **highest score**
- Global alignment (end-to-end): Needleman-Wunsch
- Local alignment: Smith-Waterman

Score matrix:

	a	c	g	t
a	+2	-2	-1	-2
c	-2	+2	-2	-1
g	-1	-2	+2	-2
t	-2	-1	-2	+2

Gap score: -3

c	c	t	a	g	t	t	c	a	t
				x		x			
t	g	t	a	a	t	-	c	a	c
		+2	+2	-1	+2	-3	+2	+2	

Alignment score = 6

Local alignments (Smith-Waterman)

- Generally useful for local alignments, determining regions of similarity between two strings (here, DNA sequence)
- Dynamic programming based algorithm: gives optimal alignment, with respect to scoring system
- Need to set scores for **match**, penalties for **mismatch** and **gap** (typically, mismatch penalties are set according to evolutionary knowledge and error profiles)
- Match: +1; Mismatch: -1; Gap: -2

Reference Sequence

	.	A	A	T	G	T
.	0	0	0	0	0	0
A	0					
T	0					
G	0					
A	0					
C	0					

$$H(i, j) = \max \begin{cases} 0 \\ H(i-1, j-1) + w(a_i, b_j) \\ H(i-1, j) + w(a_i, -) \\ H(i, j-1) + w(-, b_j) \end{cases}, 1 \leq i \leq m, 1 \leq j \leq n$$

Match/Mismatch

Deletion

Insertion

Local alignments (Smith-Waterman)

- Generally useful for local alignments, determining regions of similarity between two strings (here, DNA sequence)
- Dynamic programming based algorithm: gives optimal alignment, with respect to scoring system
- Need to set scores for **match**, penalties for **mismatch** and **gap** (typically, mismatch penalties are set according to evolutionary knowledge and error profiles)
- Match: +1; Mismatch: -1; Gap: -2

	.	A	A	T	G	T
.	0	0	0	0	0	0
A	0	1	1			
T	0					
G	0					
A	0					
C	0					

$$H(i, j) = \max \begin{cases} 0 \\ H(i-1, j-1) + w(a_i, b_j) \\ H(i-1, j) + w(a_i, -) \\ H(i, j-1) + w(-, b_j) \end{cases}, 1 \leq i \leq m, 1 \leq j \leq n$$

Match/Mismatch

Deletion

Insertion

Local alignments (Smith-Waterman)

- Generally useful for local alignments, determining regions of similarity between two strings (here, DNA sequence)
- Dynamic programming based algorithm: gives optimal alignment, with respect to scoring system
- Need to set scores for **match**, penalties for **mismatch** and **gap** (typically, mismatch penalties are set according to evolutionary knowledge and error profiles)
- Match: +1; Mismatch: -1; Gap: -2

	.	A	A	T	G	T
.	0	0	0	0	0	0
A	0	1	1	0	0	0
T	0	0	0	2	0	1
G	0					
A	0					
C	0					

$$H(i, j) = \max \begin{cases} 0 \\ H(i-1, j-1) + w(a_i, b_j) & \text{Match/Mismatch} \\ H(i-1, j) + w(a_i, -) & \text{Deletion} \\ H(i, j-1) + w(-, b_j) & \text{Insertion} \end{cases}, 1 \leq i \leq m, 1 \leq j \leq n$$

Local alignments (Smith-Waterman)

- Generally useful for local alignments, determining regions of similarity between two strings (here, DNA sequence)
- Dynamic programming based algorithm: gives optimal alignment, with respect to scoring system
- Need to set scores for **match**, penalties for **mismatch** and **gap** (typically, mismatch penalties are set according to evolutionary knowledge and error profiles)
- Match: +1; Mismatch: -1; Gap: -2

	.	A	A	T	G	T
.	0	0	0	0	0	0
A	0	1	1	0	0	0
T	0	0	0	2	0	1
G	0	0	0	0	3	1
A	0	1	1	0	1	2
C	0	0	0	0	0	0

$$H(i, j) = \max \begin{cases} 0 \\ H(i-1, j-1) + w(a_i, b_j) \\ H(i-1, j) + w(a_i, -) \\ H(i, j-1) + w(-, b_j) \end{cases}, 1 \leq i \leq m, 1 \leq j \leq n$$

Match/Mismatch

Deletion

Insertion

Local alignments (Smith-Waterman)

- Traceback:
- Start with maximum score (here, 3)
- Follow path that gives maximum score

	.	A	A	T	G	T
.	0	0	0	0	0	0
A	0	1	1	0	0	0
T	0	0	0	2	0	1
G	0	0	0	0	3	1
A	0	1	1	0	1	2
C	0	0	0	0	0	0

$$H(i, j) = \max \begin{cases} 0 \\ H(i-1, j-1) + w(a_i, b_j) \\ H(i-1, j) + w(a_i, -) \\ H(i, j-1) + w(-, b_j) \end{cases}, 1 \leq i \leq m, 1 \leq j \leq n$$

Match/Mismatch

Deletion

Insertion

Local alignments (Smith-Waterman)

- Traceback:
- Start with maximum score (here, 3)
- Follow path that gives multiple score
- This gives:

ref : aATGt
read: ATGac

lower case = not part of the local alignment

	.	A	A	T	G	T
.	0	0	0	0	0	0
A	0	1	1	0	0	0
T	0	0	0	2	0	1
G	0	0	0	0	3	1
A	0	1	1	0	1	2
C	0	0	0	0	0	0

$$H(i, j) = \max \begin{cases} 0 \\ H(i-1, j-1) + w(a_i, b_j) \\ H(i-1, j) + w(a_i, -) \\ H(i, j-1) + w(-, b_j) \end{cases}, 1 \leq i \leq m, 1 \leq j \leq n$$

Match/Mismatch

Deletion

Insertion

Global Alignment With Affine Gap Penalties

- Other considerations:
- Natural extension is to have a different *gap opening* and a *gap extension* penalty (former generally being larger)
- GO penalty=-2; GE penalty=-1; Mismatch=-1; Match=1
- With above penalties, which is the best scoring alignment?

- | | | |
|-----------|---------|----------|
| • AT-C-GT | ATC--GT | AT-C--GT |
| • ATTTTGT | ATTTTGT | ATT-TTGT |

Answer: One Long Gap gets higher Score than two Short Gaps

a)

A	T	-	C	-	G	T	
A	T	T	T	T	G	T	
+1	+1	-2	-1	-2	+1	+1	= -1

b) maximal score

A	T	C	-	-	G	T	
A	T	T	T	T	G	T	
+1	+1	-1	-2	-1	+1	+1	= 0

c)

A	T	-	C	-	-	G	T
A	T	T	-	T	T	G	T
+1	+1	-2	-2	-2	-1	+1	+1 = -3

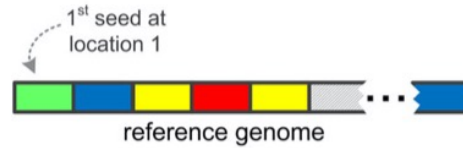
- One gap of length 2 give a penalty of -3; two gaps of length 1 cost -4
- Scores are log-odds: an affine penalty is a geometric model of indel length — opening a gap is the rare event, extending it is cheap
- Alignment representation is not unique: A--TCGT and AT--CGT also score 0. Where an indel is placed is arbitrary — which is why variant callers left-align indels

Alignment Tools

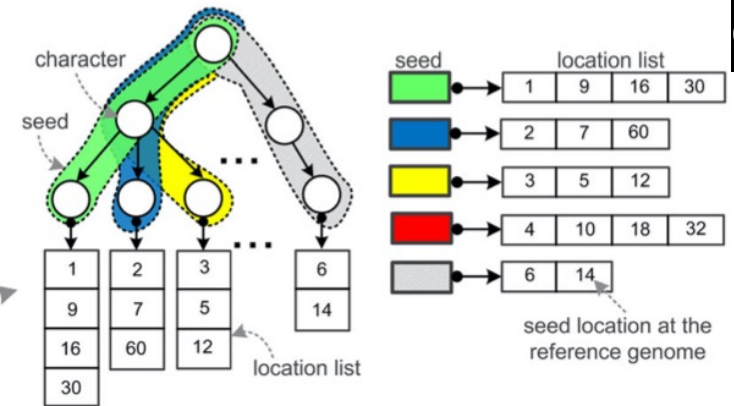
- Dynamic programming is slow
- Speed-up with heuristics
 - e.g. exactly align short subsequences and extend these alignments
 - e.g. BLAST / BLAT
 - no longer guaranteed to find the best alignments
 - exact matches are found by index lookup

Hashing based alignment

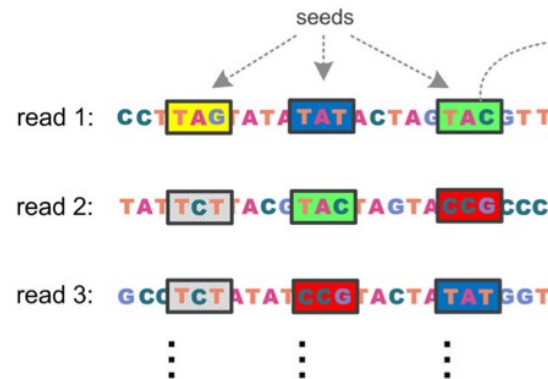
a. Seed extraction from reference genome



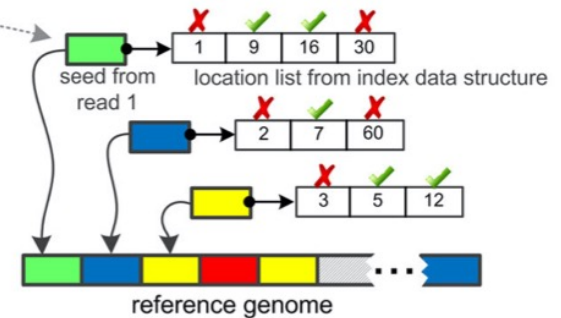
b. Seed indexing using suffix tree or hash table



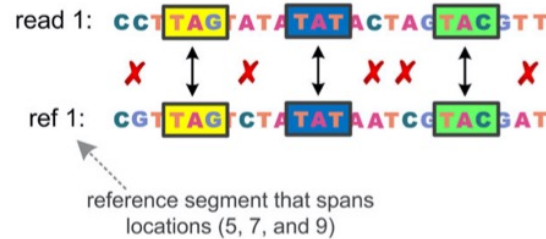
c. Seed extraction from reads



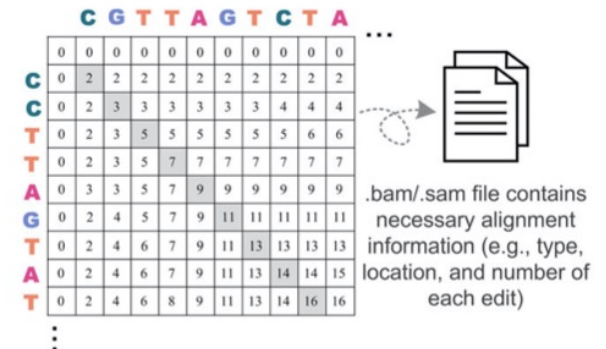
d. Seed querying and filtering



e. Seed chaining and pre-alignment filtering

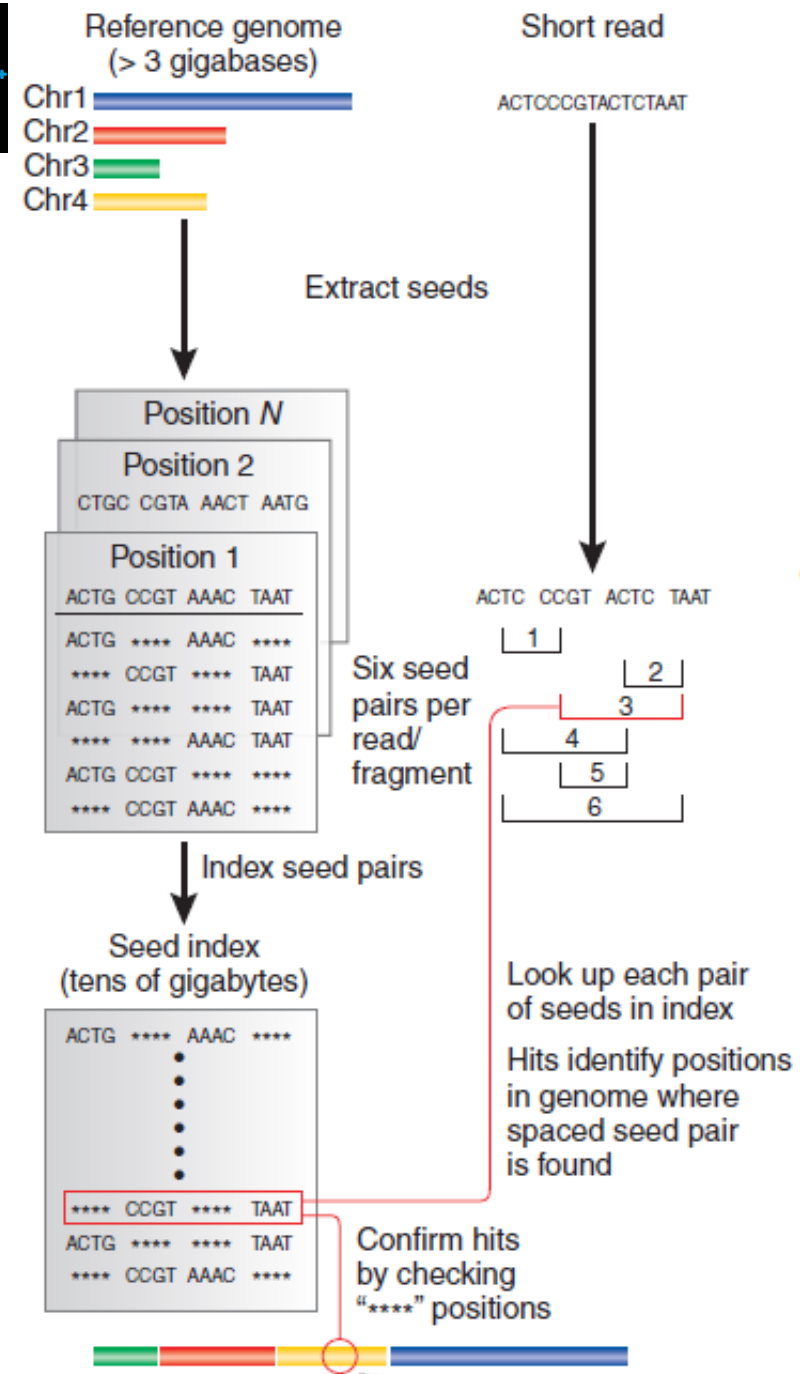


f. Alignment verification



Index Genome: Spaced Seeds

- Tags and tag-sized pieces of reference are cut into small “seeds.”
- Pairs of spaced seeds are stored in an index.
- Look up spaced seeds for each tag.
- For each “hit,” confirm the remaining positions.

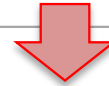


The Burrows-Wheeler transform (1994; 1983)

c a c a a c g \$



c	a	c	a	a	c	g	\$
a	c	a	a	c	g	\$	c
c	a	a	c	g	\$	c	a
a	a	c	g	\$	c	a	c
a	c	g	\$	c	a	c	a
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c
\$	c	a	c	a	a	c	g



\$	c	a	c	a	a	c	g
a	a	c	g	\$	c	a	c
a	c	a	a	c	g	\$	c
a	c	g	\$	c	a	c	a
c	a	a	c	g	\$	c	a
c	a	c	a	a	c	g	\$
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c



g c c a a \$ a c

The “Last-First mapping” property

The relative ordering of a particular character (say **c**) in column 1 is the same as that in the last column

c_1 a c_2 a a c_3 g \$

\$	c_1	a	c_2	a	a	c_3	g
a	a	c_3	g	\$	c_1	a	c_2
a	c_2	a	a	c_3	g	\$	c_1
a	c_3	g	\$	c_1	a	c_2	a
c_2	a	a	c_3	g	\$	c_1	a
c_1	a	c_2	a	a	c_3	g	\$
c_3	g	\$	c_1	a	c_2	a	a
g	\$	c_1	a	c_2	a	a	c_3

The “Last-First mapping” property

The relative ordering of a particular character (say **c**) in column 1 is the same as that in the last column

c_1 a c_2 a a c_3 g \$

\$	c_1	a	c_2	a	a	c_3	g
a	a	c_3	g	\$	c_1	a	c_2
a	c_2	a	a	c_3	g	\$	c_1
a	c_3	g	\$	c_1	a	c_2	a
c_2	a	a	c_3	g	\$	c_1	a
c_1	a	c_2	a	a	c_3	g	\$
c_3	g	\$	c_1	a	c_2	a	a
g	\$	c_1	a	c_2	a	a	c_3

Proof:

Suppose cX and cY are cyclic permutations of the input T .

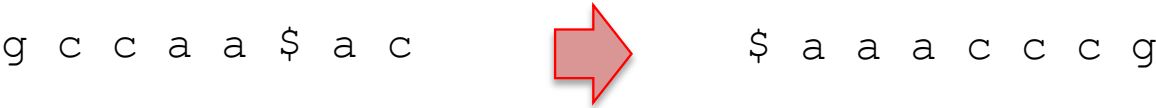
Suppose $cX < cY$ (in lexicographical ordering)

Then $Xc < Yc$ (in lexicographical ordering)

The LF-mapping property follows.

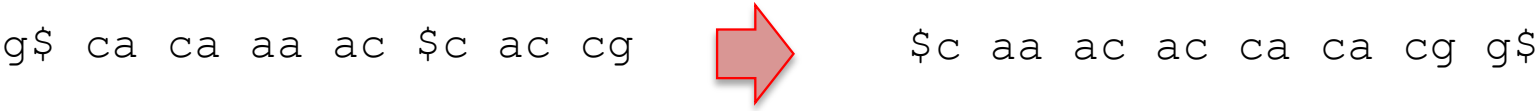
BWT is reversible

\$	c	a	c	a	a	c	g
a	a	c	g	\$	c	a	c
a	c	a	a	c	g	\$	c
a	c	g	\$	c	a	c	a
c	a	a	c	g	\$	c	a
c	a	c	a	a	c	g	\$
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c



BWT is reversible

\$	c	a	c	a	a	c	g
a	a	c	g	\$	c	a	c
a	c	a	a	c	g	\$	c
a	c	g	\$	c	a	c	a
c	a	a	c	g	\$	c	a
c	a	c	a	a	c	g	\$
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c



BWT is reversible

\$	c	a	c	a	a	c	g
a	a	c	g	\$	c	a	c
a	c	a	a	c	g	\$	c
a	c	g	\$	c	a	c	a
c	a	a	c	g	\$	c	a
c	a	c	a	a	c	g	\$
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c

Lookup AAC

\$	c	a	c	a	a	c	g
a	a	c	g	\$	c	a	c
a	c	a	a	c	g	\$	c
a	c	g	\$	c	a	c	a
c	a	a	c	g	\$	c	a
c	a	c	a	a	c	g	\$
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c

Range $\leftarrow$ range of last character in 1st column

While characters left (and nonzero range):

Lookup first and last match to preceding character in final column

Range $\leftarrow$ LF-mapping of first and last match

Lookup AAC

\$	c	a	c	a	a	c	g
a	a	c	g	\$	c	a	c
a	c	a	a	c	g	\$	c
a	c	g	\$	c	a	c	a
c	a	a	c	g	\$	c	a
c	a	c	a	a	c	g	\$
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c

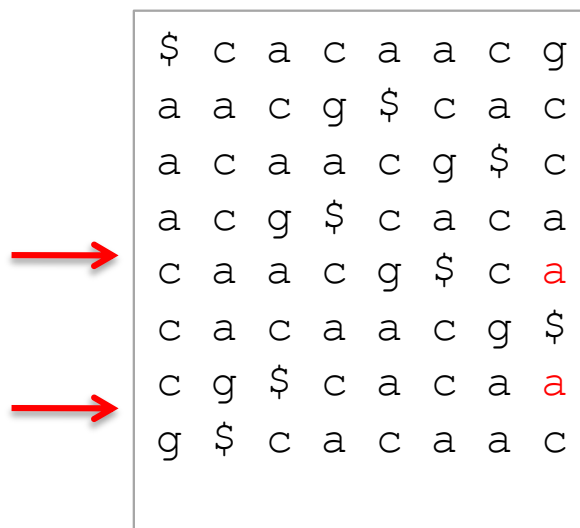
Range $\leftarrow$ range of last character in 1st column

While characters left (and nonzero range):

Lookup first and last match to preceding character in final column

Range $\leftarrow$ LF-mapping of first and last match

Lookup A**C**



\$	c	a	c	a	a	c	g
a	a	c	g	\$	c	a	c
a	c	a	a	c	g	\$	c
a	c	g	\$	c	a	c	a
c	a	a	c	g	\$	c	a
c	a	c	a	a	c	g	\$
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c

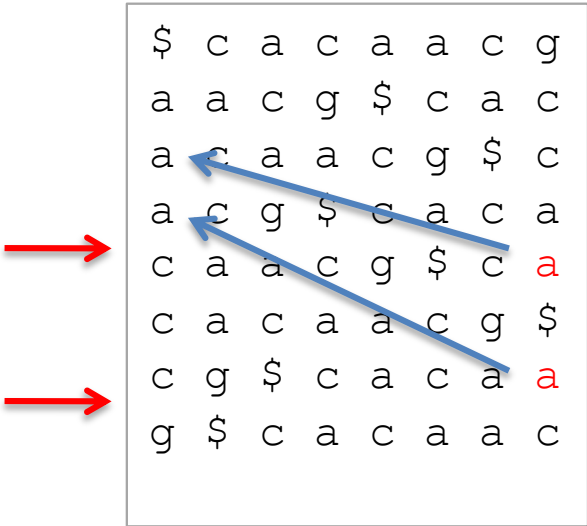
Range $\leftarrow$ range of last character in 1st column

While characters left (and nonzero range):

Lookup first and last match to preceding character in final column

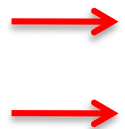
Range $\leftarrow$ LF-mapping of first and last match

Lookup A**AC**



Range ← range of last character in 1st column
While characters left (and nonzero range):
 Lookup first and last match to preceding character in final column
 Range ← LF-mapping of first and last match

Lookup A**A**C



\$	c	a	c	a	a	c	g
a	a	c	g	\$	c	a	c
a	c	a	a	c	g	\$	c
a	c	g	\$	c	a	c	a
c	a	a	c	g	\$	c	a
c	a	c	a	a	c	g	\$
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c

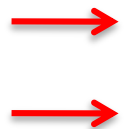
Range $\leftarrow$ range of last character in 1st column

While characters left (and nonzero range):

Lookup first and last match to preceding character in final column

Range $\leftarrow$ LF-mapping of first and last match

Lookup AAC



\$	c	a	c	a	a	c	g
a	a	c	g	\$	c	a	c
a	c	a	a	c	g	\$	c
a	c	g	\$	c	a	c	a
c	a	a	c	g	\$	c	a
c	a	c	a	a	c	g	\$
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c

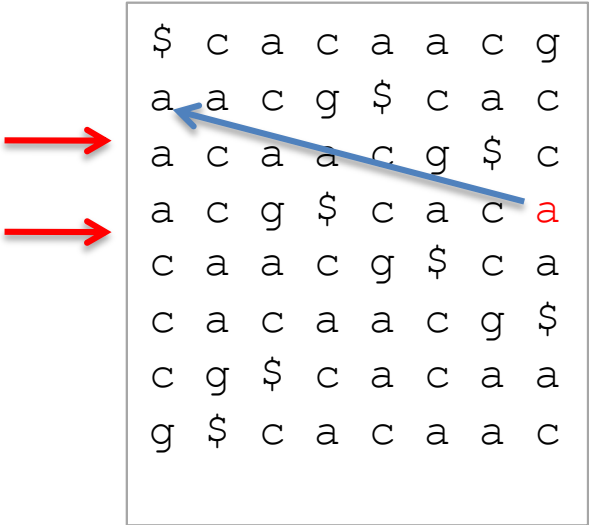
Range $\leftarrow$ range of last character in 1st column

While characters left (and nonzero range):

Lookup first and last match to preceding character in final column

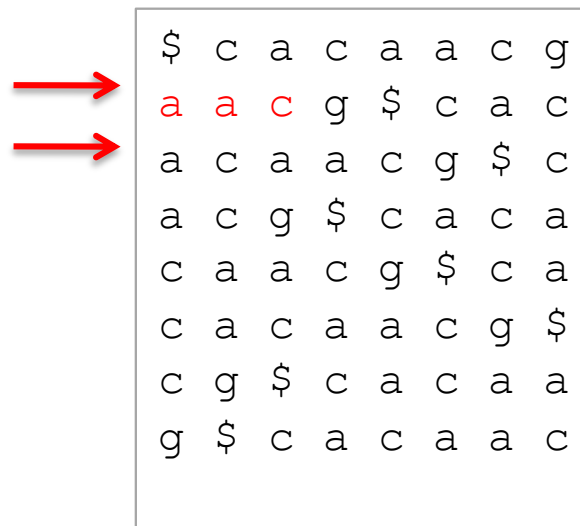
Range $\leftarrow$ LF-mapping of first and last match

Lookup **A**AC



Range $\leftarrow$ range of last character in 1st column
 While characters left (and nonzero range):
 Lookup first and last match to preceding character in final column
 Range $\leftarrow$ LF-mapping of first and last match

Lookup AAC



\$	c	a	c	a	a	c	g
a	a	c	g	\$	c	a	c
a	c	a	a	c	g	\$	c
a	c	g	\$	c	a	c	a
c	a	a	c	g	\$	c	a
c	a	c	a	a	c	g	\$
c	g	\$	c	a	c	a	a
g	\$	c	a	c	a	a	c

Range $\leftarrow$ range of last character in 1st column

While characters left (and nonzero range):

Lookup first and last match to preceding character in final column

Range $\leftarrow$ LF-mapping of first and last match

Comparison

Spaced seeds

- Memory depends on the index design — Subread maps to human with ≈ 8 GB (gapped index) or ≈ 18 GB (full index)
- More straightforward to program
- Example: Subread / Subjunc
- More tolerant of
 - sequence variation
 - sequencing errors

Burrows-Wheeler

- Compressed into an **FM-index** ('Full-text index in Minute space', Ferragina & Manzini, 2000)
- Mapping to human needs ≈ 3.2 GB (bowtie2) to ≈ 5 – 6 GB (BWA) of RAM; the famous "< 2 GB" was Bowtie 1 (2009)
- Can be very fast
- More complicated to program
- Classical tools: bowtie2, BWA

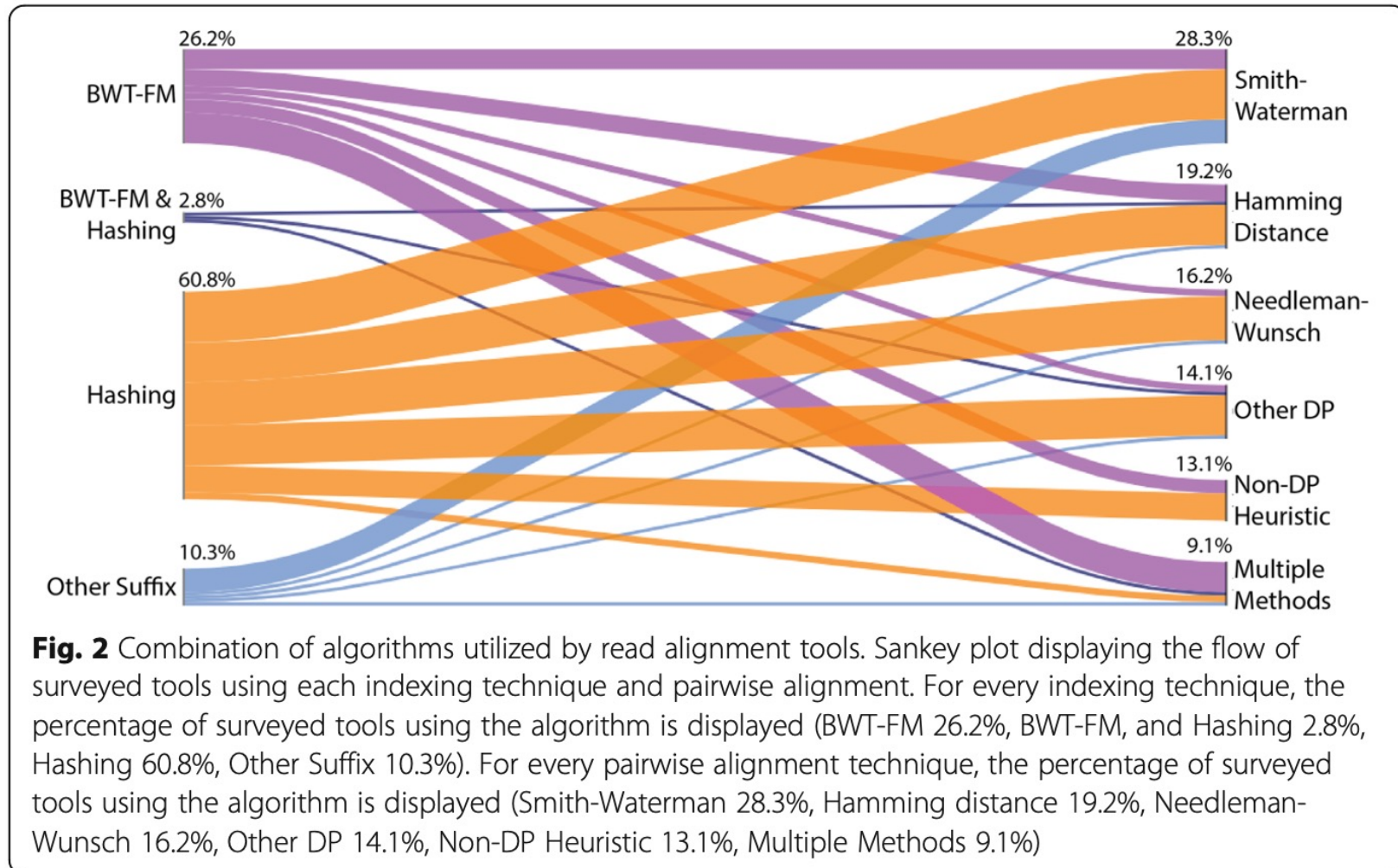
Characteristics of algorithms

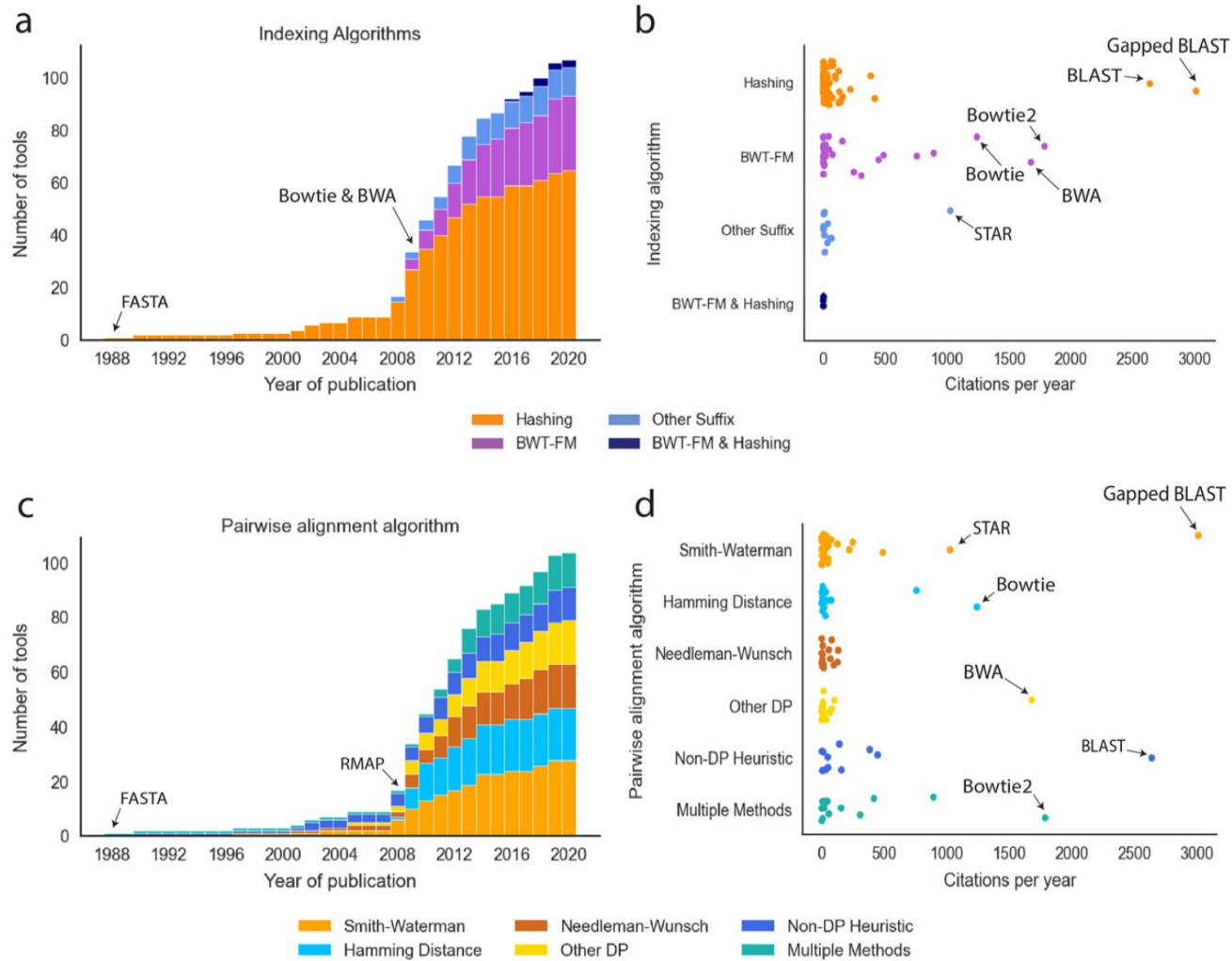
	Hashing	Suffix tree and BWT-FM
Easy to implement	Yes	No
Search for exact/inexact match	Exact	Exact and inexact
Index size	Large	Compressed (small)
Indexing time	Small	Large
Seed query speed	O(1), fast	Slow
Seed length	Fixed length per index	Can be fixed or variable

Alser et al.

<https://genomebiology.biomedcentral.com/articles/10.1186/s13059-021-02443-7>

Combining index lookup and pairwise alignment





Alignment with Mismatches

- Mismatches can occur because of
 - sequencing error (error rate $\sim 1/1000$)
 - genetic variation: a genome differs from the reference at 4–5 million sites (≈ 1 in 700 bp)
 - $\rightarrow$ together, $\approx 20\%$ of 100-nt reads and $\approx 25\text{--}30\%$ of 150-nt reads carry at least one mismatch
- Above numbers are for the whole DNA, how about coding sequences in RNA?
 - coding DNA is more conserved due to purifying selection $\rightarrow$ about 1 in 1500 bp differ
- If there is a sequence mismatch the index lookup fails! How to find nevertheless an alignment?

BWT Alignment with Mismatches

- Strategies:
 - If the BWT lookup fails at position k, try all different bases at position k
→ drawback: computing effort grows exponentially with the number of mismatches (every mismatch expands the queries by a factor of 4)
→ implemented e.g. in Bowtie 1 (mismatches only, no gaps)
 - Chop reads in segments (seeds) and align those mismatch-free and stitch seed alignments together
→ implemented e.g. in bowtie2 and BWA-MEM (seed-and-extend, gaps allowed)

Consideration: Sequencing Errors – PHRED quality

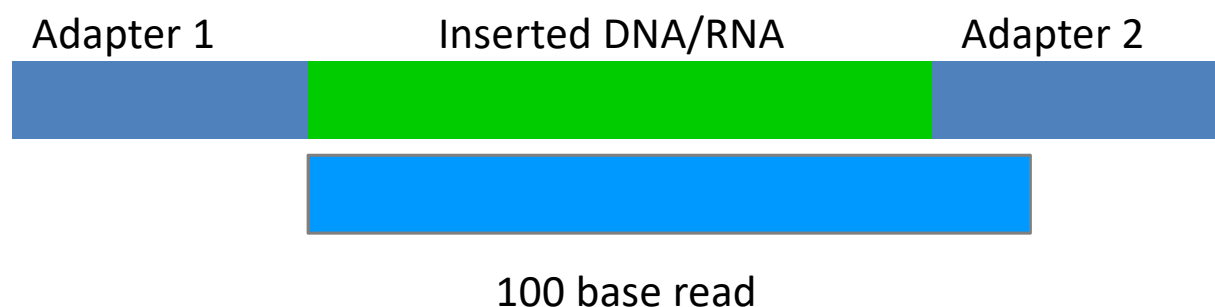
- Each called base is given a quality score Q estimated by the base-calling software:
- $Q = -10 * \log_{10}(\text{estimated probability that call is wrong})$
 - 10 prob = 0.1
 - 20 prob = 0.01
 - 30 prob = 0.001

→ down-weight low-quality bases when computing the alignment score

Considerations: Read trimming

- Sequencers have systematic errors
- Illumina sequencers have a higher error rate at the first few bases (1-5)
- Basically, all sequencers have increasing error rate towards the end of the read
- Hard trimming: trim a fixed number of bases from the beginning and/or end
- Quality trimming: cut the end of the read as soon as the base quality drops below a threshold

Considerations: Read trimming



- If the inserted DNA/RNA fragment is too short, the read will contain part of the adapter
- Adapter trimming can be challenging if
 - if the insert is 90 – 100 bases
 - if the read has many sequencing errors

Multiple Alignments

- A read may have multiple valid alignments with identical or similarly good alignment scores
- Aligners allow to choose different reporting strategies:
 - Randomly select one alignment from the top-scoring alignments
 - Report all alignments that are within *delta* of the top-scoring alignment; clip if more than *Nmax* alignments are found
 - Report only alignments if they are unique (no other alignment within *delta* of the alignment score)
 - Do not report anything if more than *Nmax* valid alignments are found
 - ...
- Whether a read has a **unique alignment** depends on
 - the read sequence and the sequence homology of the organism
 - the search algorithm of the aligner
 - the reporting strategy of the aligner

Read Alignment Summary (I)

How to map billions of reads?

- Many aligners index the reference with the Burrows-Wheeler transform (FM-index); for the human genome that index needs about 3–6 GB of RAM (STAR's suffix array: ~32 GB).
- Reads are aligned by **index lookup** not by sequence comparison
- The lookup of a perfect match read is faster than loading the read and writing the alignment coordinates to the output file!
- If the lookup fails because of SNPs or sequencing errors, an actual sequence comparison is done – and this can take time!

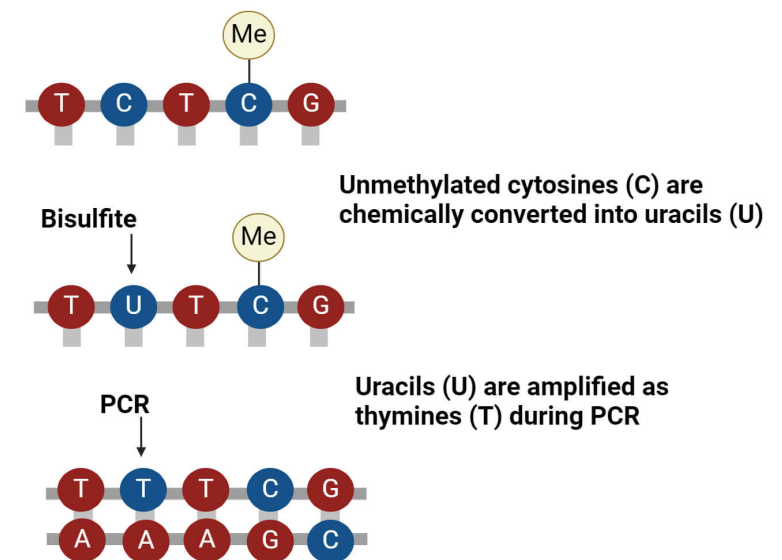
Read Alignment Summary (II)

- Alignment is a score optimization problem
- Aligners find the alignment with the **highest alignment score**
- Aligners do not run an extensive search, they use **index lookup and heuristics** to find the alignment with highest score; these heuristics are not guaranteed to find the alignment with the maximum score
- If the heuristics are well chosen and the scores are well defined, then the alignment will be very often the highest-scoring alignment and that will be the correct alignment

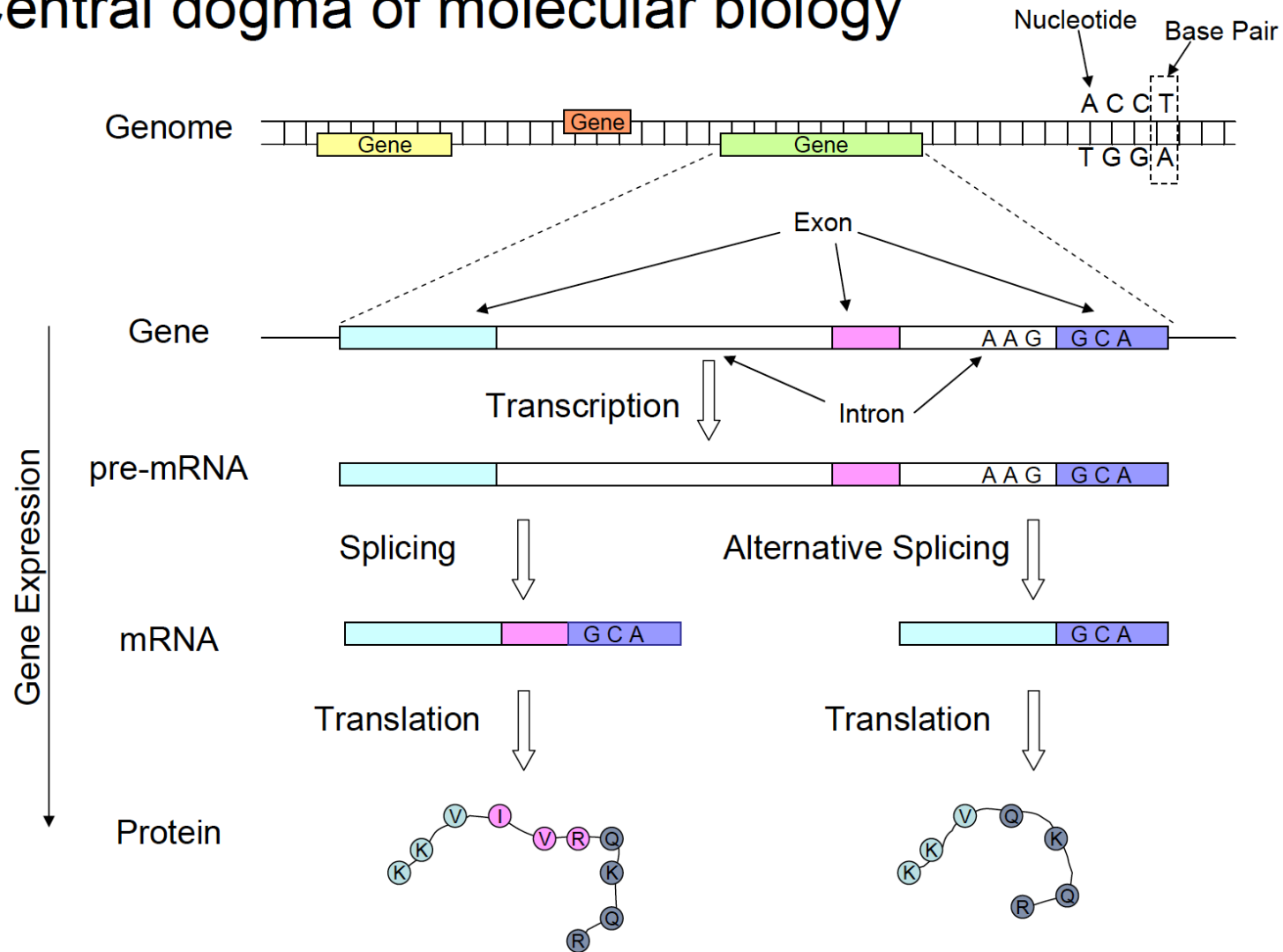
Adapted mapping strategies do exist for special applications

- **DNA samples:** “standard” mapping using whole genome sequence as target
- **RNA samples:** account for RNA splicing
- **microRNAs:** short fragments (19-24 bases); map to micro-RNA database *mirBase*
- **Bisulfite sequencing:** needs DNA methylation-aware aligners (e.g., Bismark, BSMAP).
- **ATAC-seq / ChIP-seq:** local alignment with attention to soft clipping and duplicate removal
- **Long reads:** account for larger divergence

Bisulfite Conversion



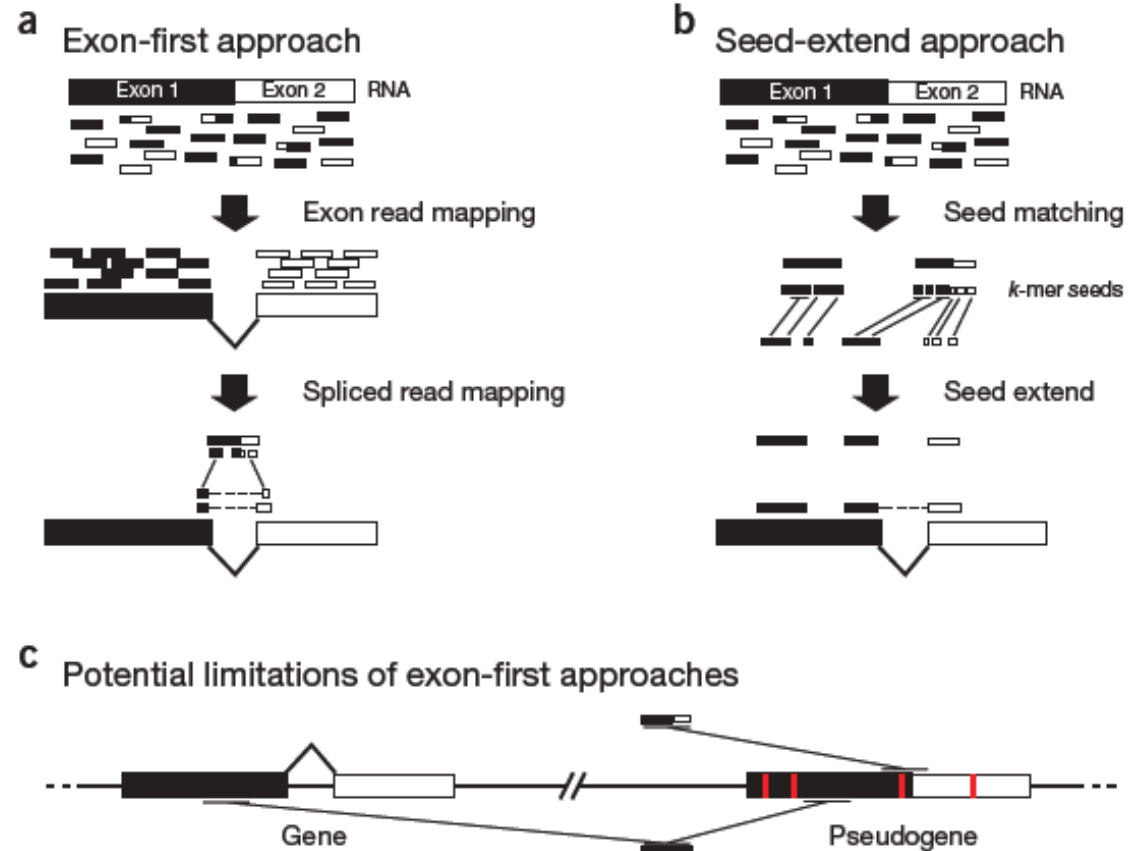
Central dogma of molecular biology



92–94% of human genes undergo alternative splicing,

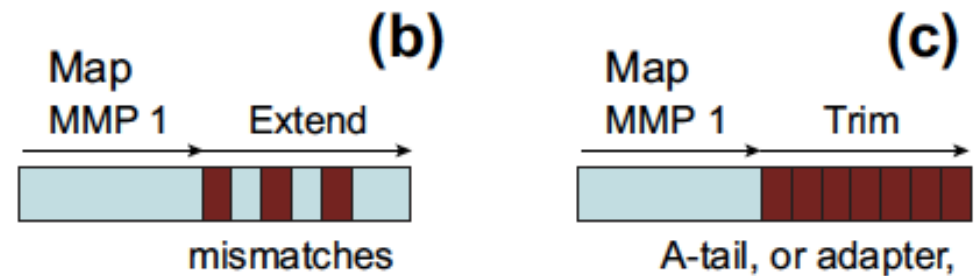
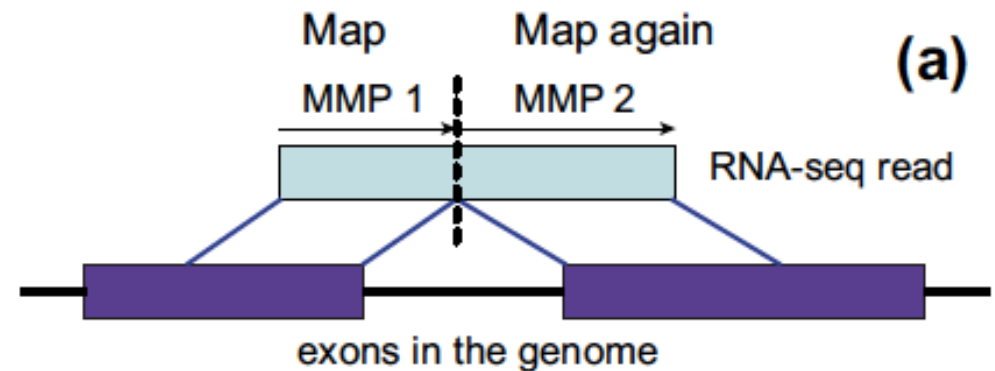
RNA-seq Mapping

- Mapping targets:
 - transcriptome
 - splice junction library
 - genome
- Exon-first approaches are biased to find unspliced matches

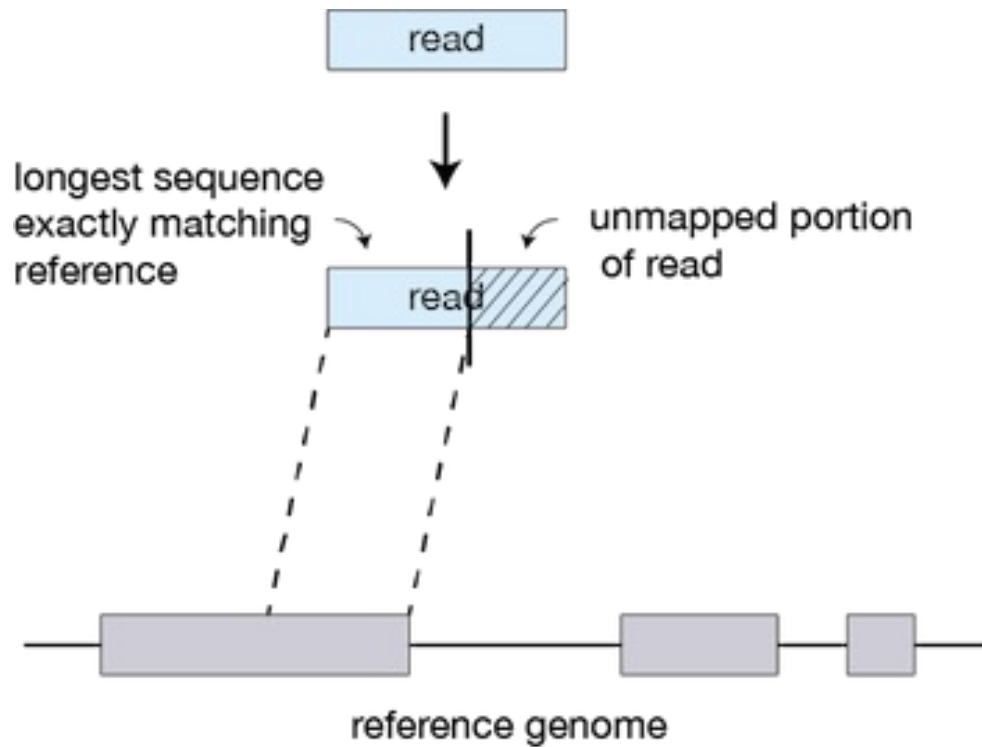


STAR: universal RNA-seq aligner

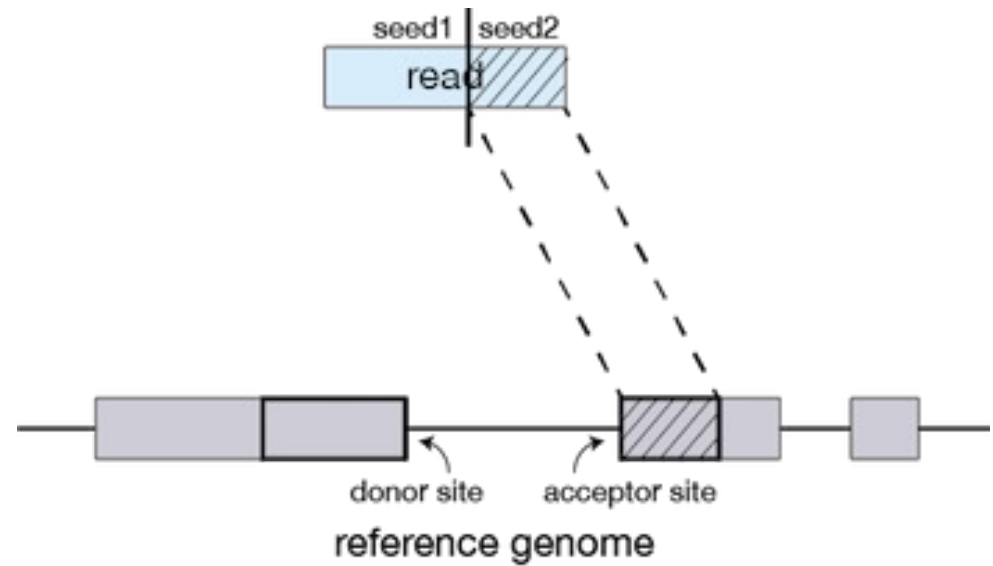
- Designed to align the non-contiguous sequences directly to the reference genome
- Steps:
 - Search Maximal Mappable Prefix (MMP)
 - clustering/stitching/scoring



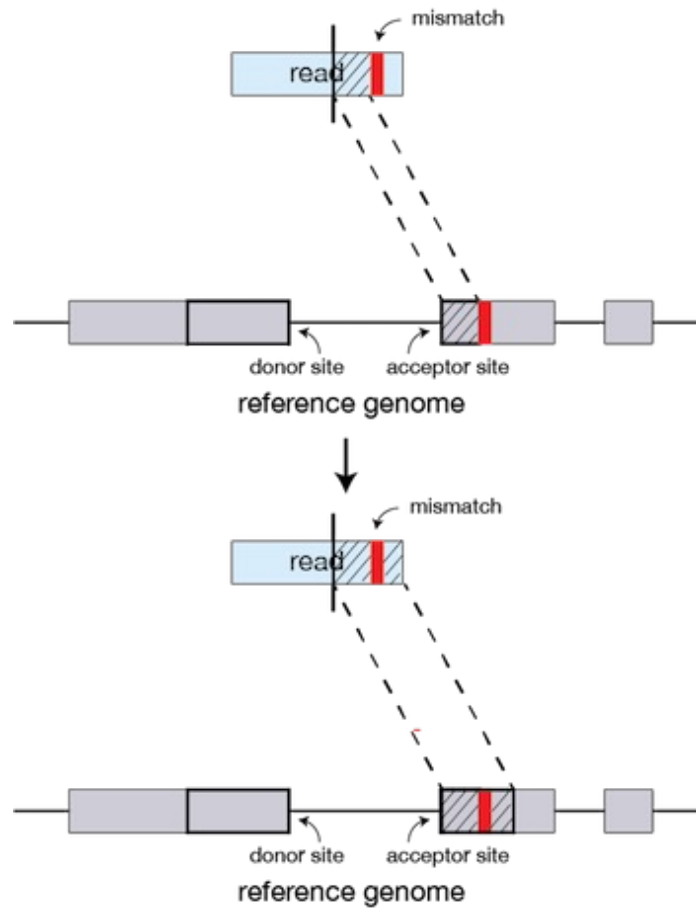
Seed lookup



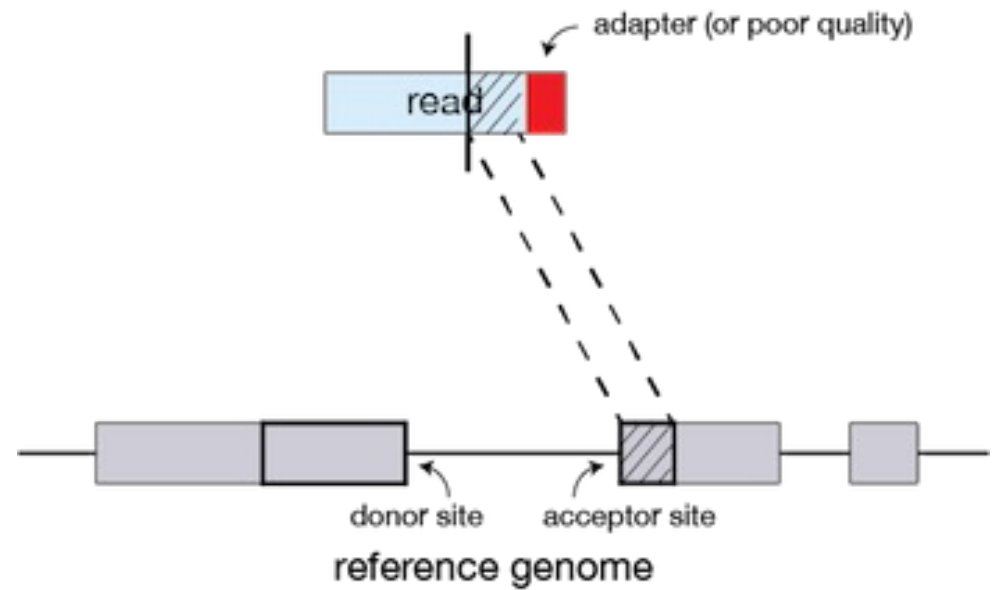
Reseed



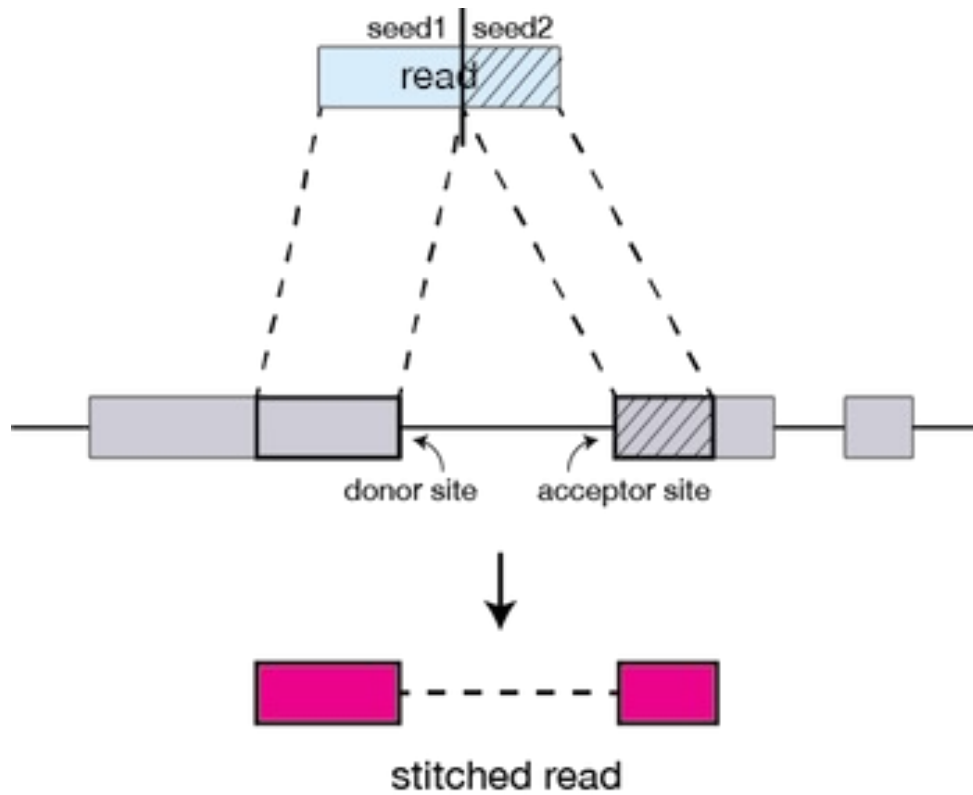
Handle mismatches



Soft clip ends



Stitch spliced



STAR input

- STAR can align to genome sequence and finds also splice junctions
- Performance is better if gene coordinates (gtf file) is provided
- STAR will build an index that holds
 - Genome sequence
 - Splice junction sequences

Comparison of Aligners

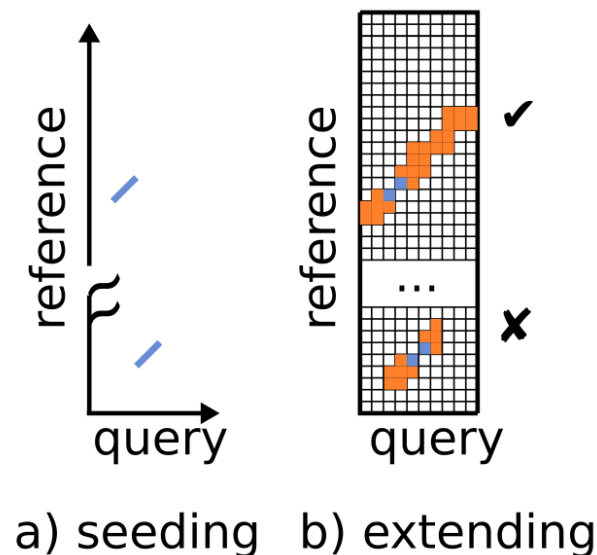
- Tradeoff:
 - speed
 - RAM
 - accuracy
 - sensitivity
- Most popular tools:
 - STAR
 - HISAT2
 - Subread/Subjunc

STAR 2-pass mode

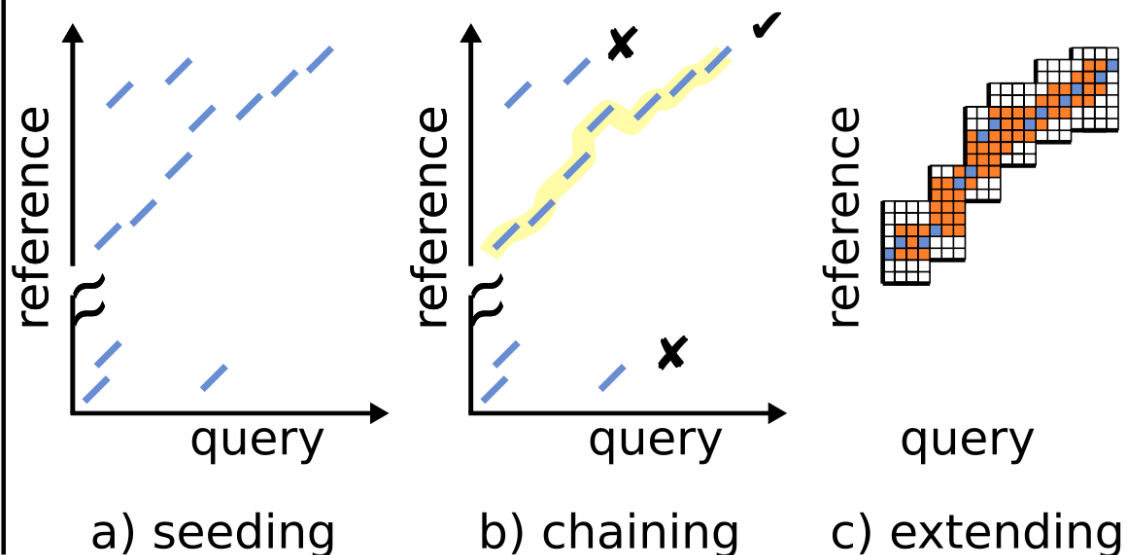
- goal: improve alignment around novel splice junctions
- does not discover more splice junctions but aligns more reads to discovered splice junctions
- initial mapping might fail to align reads with all bases correctly, when only a short part of the read spans the splice junction

Mapping Long Reads

short-read mapping



long-read mapping



Mainly, short-read approaches extend (orange parts) from a single anchor (in blue) on the whole read length while long-read approaches gather multiple anchors, and chain (yellow line) them in for a candidate extending procedure that is done between pairs of anchors

Mapping long reads: Minimap2

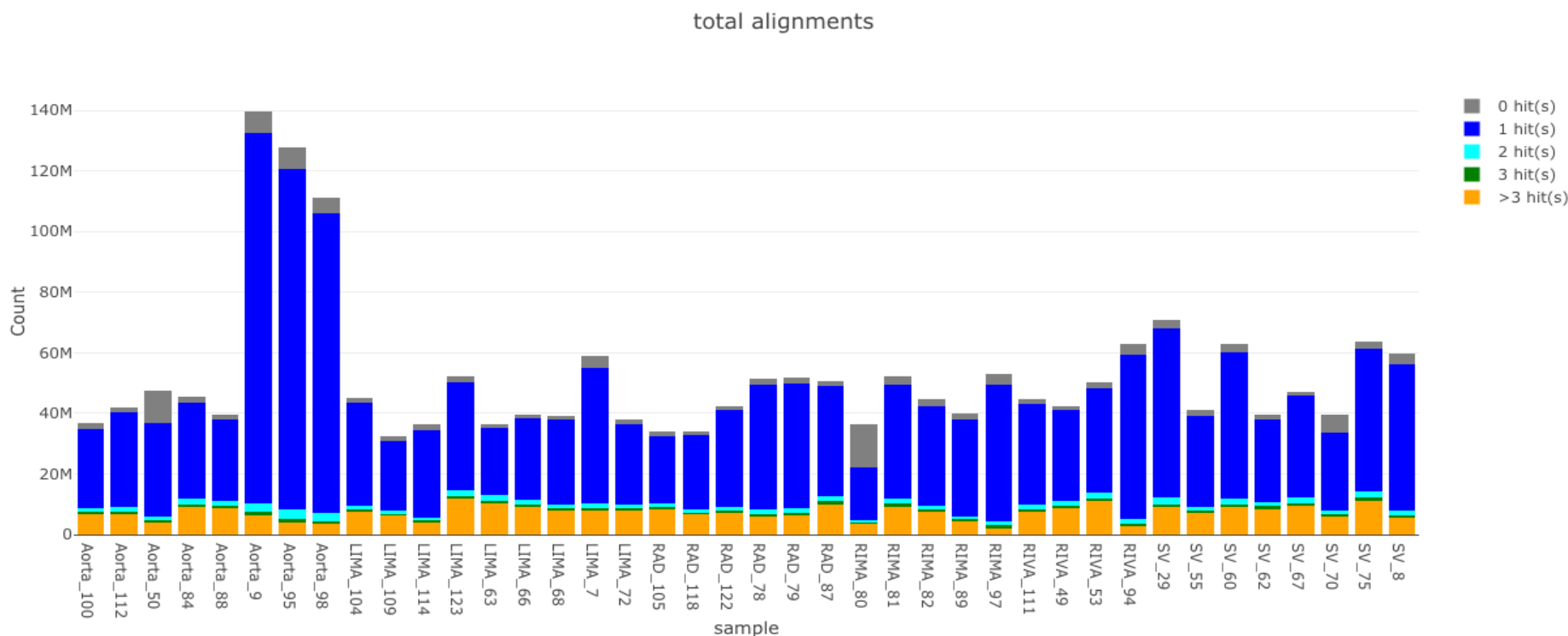
- Minimap2 uses a minimizer index:
 - A minimizer is a special kind of k-mer sampling: for each window of w consecutive k-mers in the genome, it stores the lexicographically smallest k-mer.
 - → only a fraction of all k-mers are kept
 - → reduce index size → faster search
 - Each minimizer in the reference is stored with its genomic coordinates.
- Compared to Suffix Array (STAR), FM-index (BWA) or full k-mer tables (Subread), minimizer indices use less memory and are fast at the cost of sensitivity.

How long should a read be so that it can be uniquely aligned??

Current practice

use ≥ 75 nt for aligning RNA-reads to mammals

use $\geq 100+100$ nt for aligning DNA reads for variant calling



Mappability

Mappability = fraction of the genome where a read of length k has one unique best hit

- GRCh38, $k = 24$: $\approx 81\%$ uniquely mappable; after bisulfite conversion (3-letter alphabet) only $\approx 66\%$
- rises steeply with read length — but never to 100%: segmental duplications, satellites and recent gene families are longer than any short read
- two reads from one fragment help: the mate of a read inside a repeat is often unique (next two slides)
- → a gene with low mappability gets fewer unique reads at the same expression: counts are confounded with sequence content

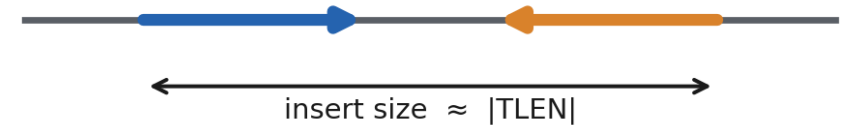
Paired-End Reads: Two Ends of One Fragment

- Both ends of one fragment are sequenced: read 1 and read 2 face each other ($\rightarrow \leftarrow$)
- The aligner places the pair jointly:
 - expected orientation and distance: the insert-size distribution is learned from confidently mapped pairs and used as a prior
- “Properly paired”: both mates aligned, facing each other, at a plausible distance
 - in SAM: FLAG bit 0x2; mate position in RNEXT / PNEXT; fragment span in TLEN
- Caveats
 - insert shorter than the read $\rightarrow$ the read runs into the adapter
 - RNA-seq: TLEN on the genome includes the introns

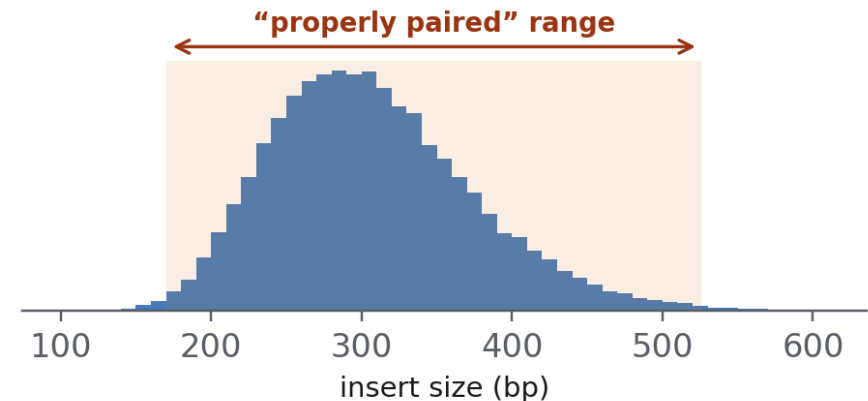
library molecule



aligned to the genome



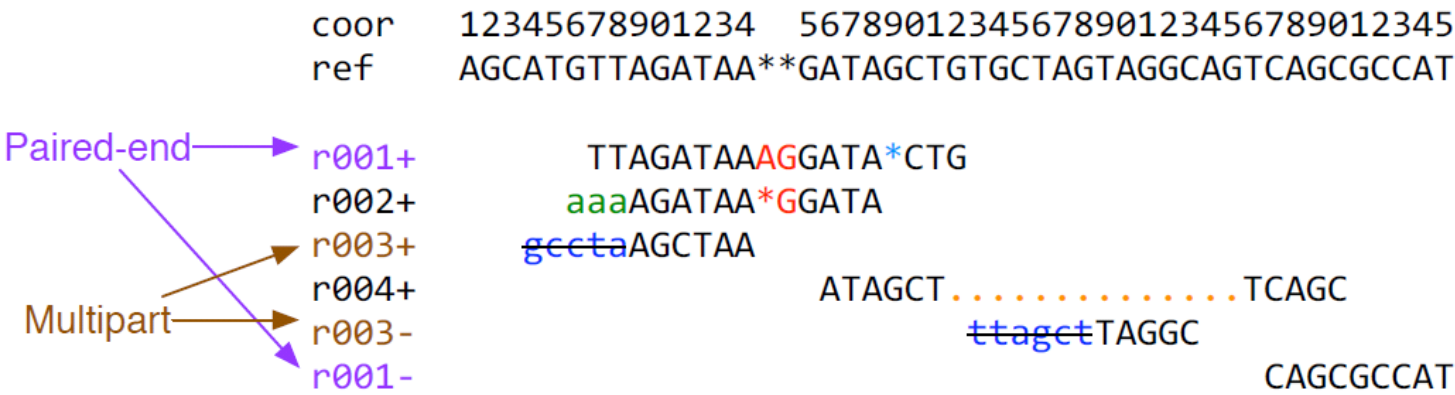
estimated from the confidently mapped pairs (illustrative)



Sequence / Alignment (SAM) files

- **SAM (Sequence Alignment/Map)**
 - Single unified format for storing read alignments to a reference genome
 - Large plain text file
 - V1.0 (2009)
 - Current specification: <https://samtools.github.io/hts-specs/>
- **BAM (Binary Alignment/Map)**
 - Binary equivalent of SAM
 - Compressed data plus index (bai)
 - Developed for fast processing/indexing
- **CRAM (Additional compression using a reference)**
 - Lossless but needs reference to reconstruct the reads

An example of read mapping



Corresponding SAM file

Paired-end

Multipart

r001+

r002+

r003+

r004+

r003-

r001-

TTAGATAAAGGATA*CTG

aaaAGATAA*GGATA

gcctaAGCTAA

ATAGCT.....TCAGC

ttagctTAGGC

CAGCGCCAT

Ins & padding

Soft clipping

Splicing

Hard clipping

@SQ SN:ref LN:45

r001 163 ref 7 30 8M2I4M1D3M = 37 39 TTAGATAAAGGATACTA *

r002 0 ref 9 30 3S6M1P1I4M * 0 0 AAAAGATAAGGATA *

r003 0 ref 9 30 5H6M * 0 0 AGCTAA * NM:i:1

r004 0 ref 16 30 6M14N5M * 0 0 ATAGCTTCAGC *

r003 16 ref 29 30 6H5M * 0 0 TAGGC * NM:i:0

r001 83 ref 37 30 9M = 7 -39 CAGCGCCAT *

QNAME

RNAME

MAPQ?

MRNM

ISIZE

QUAL

FLAG

POS

CIGAR?

MPOS

SEQ

CIGAR string - compact representation of an alignment

- M - match or mismatch
- I – insertion
- D – deletion
- S - soft clip
 - Clipped sequences stored in SAM
- H - hard clip
 - Clipped sequences not stored in SAM
- N – skipped reference bases, splicing

Match/mismatch, indels

Ref: ACGCAGTG--GT

Read: ATGCA-TGCAGT

Cigar: 5M1D2M2I2M

Soft clipping

REF: ATCGTGTAACCTGACTAGTTAA

READ: gggGTGTAACC-GACTAGggg

Cigar: 3S8M1D5M4S

Hard clipping

REF: ATCGTGTAACCTGACTAGTTAA

READ: gggGTGTAACC-GACTAGggg

Cigar: 3H8M1D5M4H

NOTE: The **M** does not mean that the base matches the reference, it means that the base contributed to the alignment and the alignment score

SAM v1.3 (2013)

Introduce the letters

- = – Sequence match with base equality
- X – Sequence mismatch
- M – becomes leacy

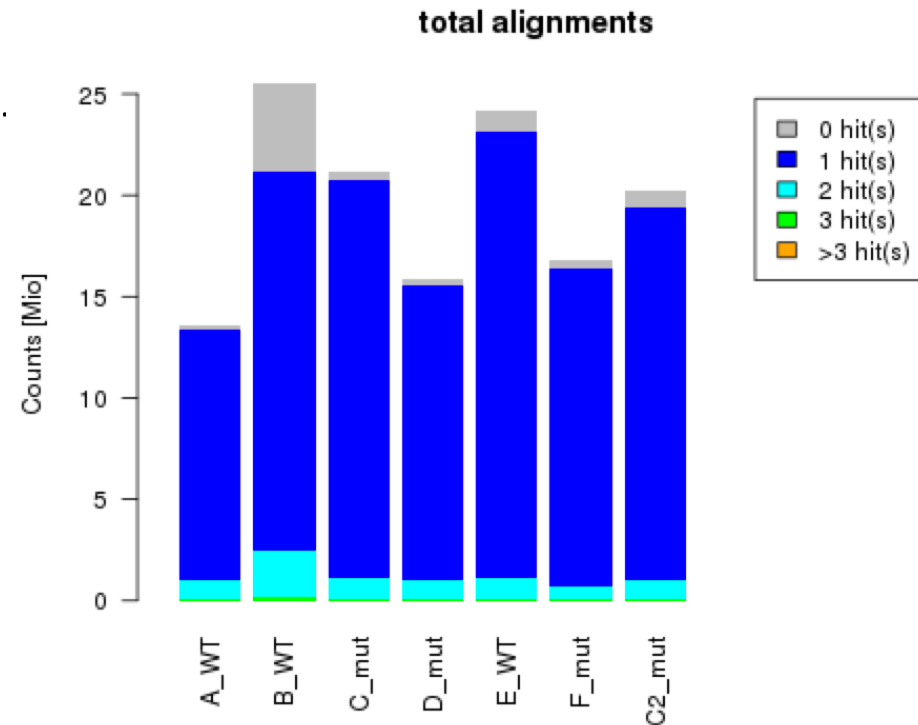
RNA-seq aligners usually still use M

Mapping quality – the MAPQ tag

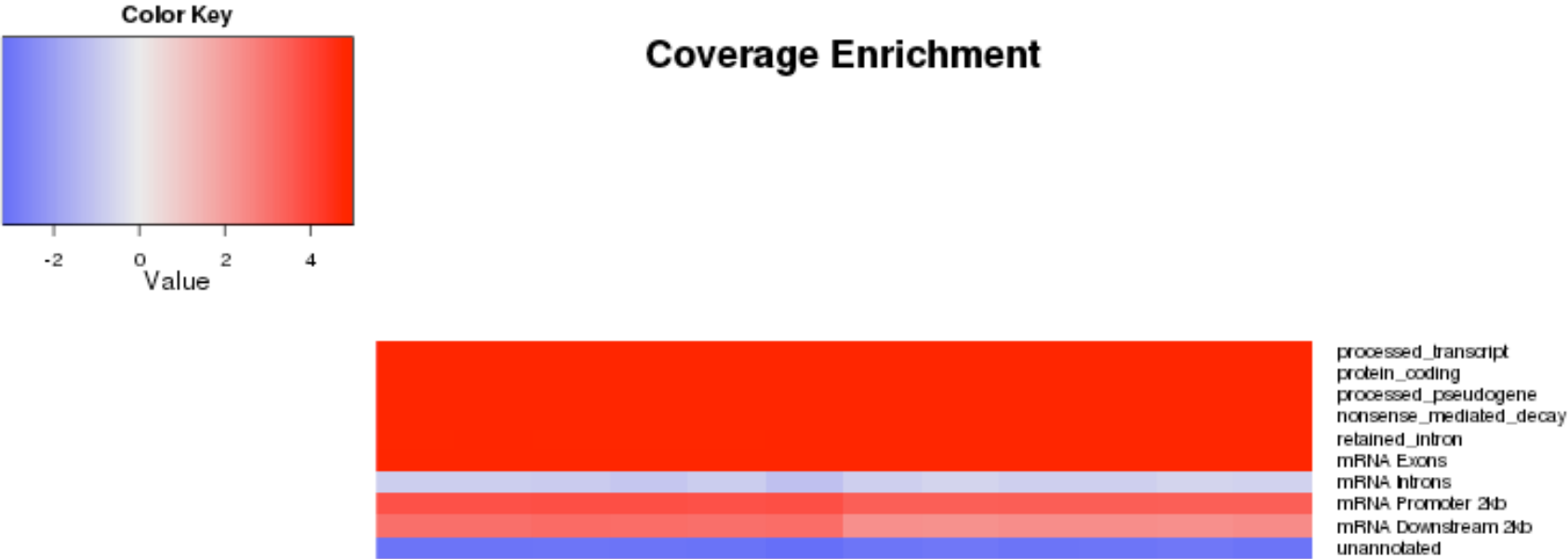
- The MAPQ tag does indicate the quality of an alignment
 - Equivalent to the PHRED score for base
 - However – the error probability can not be as accurately determined as the base quality
 - The different aligners have different methods for assigning it
 - usually from the score gap between the best and second-best hit
 - bowtie2: 0–42; BWA-MEM: 0–60
 - STAR: 255 = unique, 3 = two loci, 1 = three or four, 0 = five or more
- compare MAPQ thresholds only within one aligner

Mapping QC: Mapping statistics

- How well did my sequence library align to my reference?
- Summary statistics (per read library)
 - % reads with unique alignment
 - % reads with multiple alignment
 - % reads with no alignment
 - % reads properly paired (for paired end libraries)



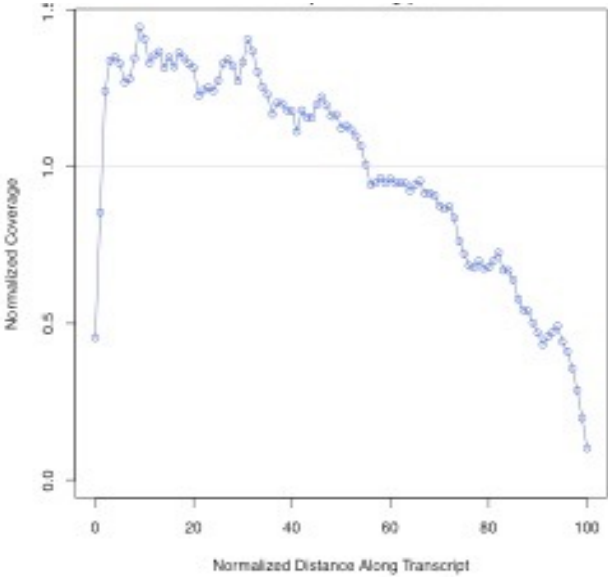
Mapping QC: Profiling efficiency by transcript annotation



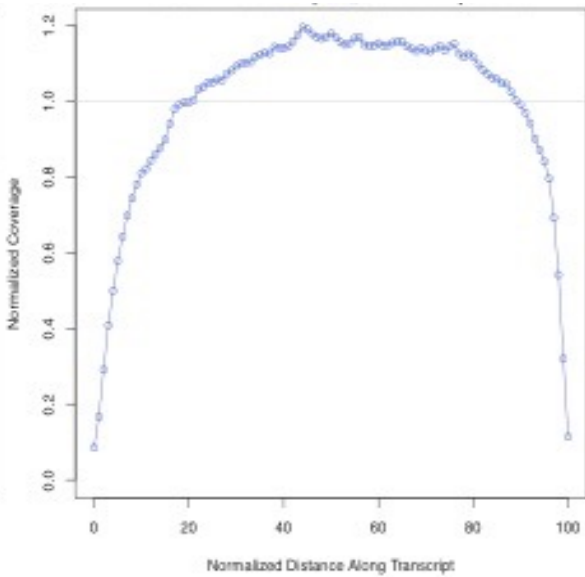
- Transcript annotation: intron, exon, up/down stream, unannotated
- Expression profiling efficiency

Mapping QC: Coverage bias

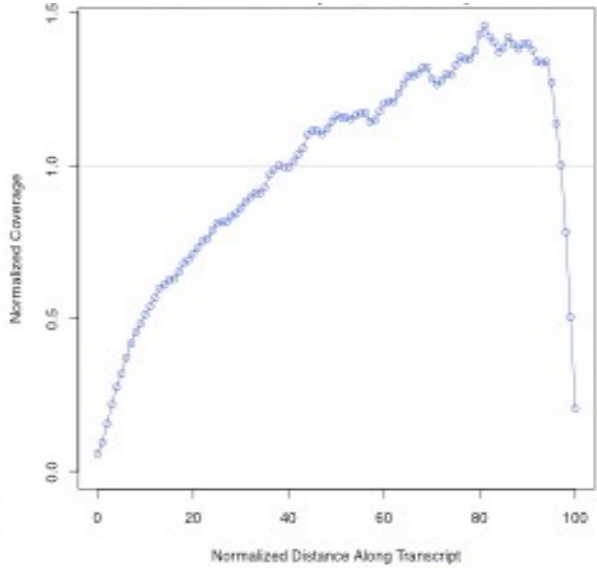
3' bias



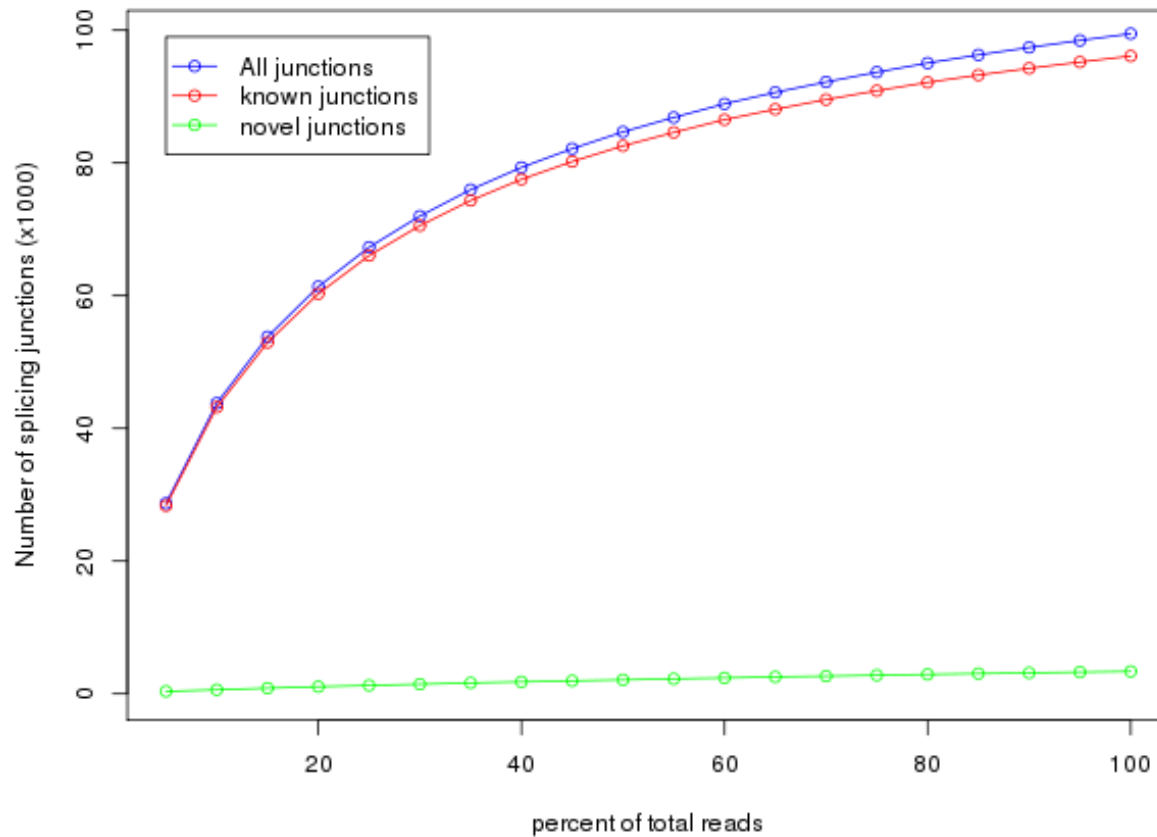
No bias



5' bias

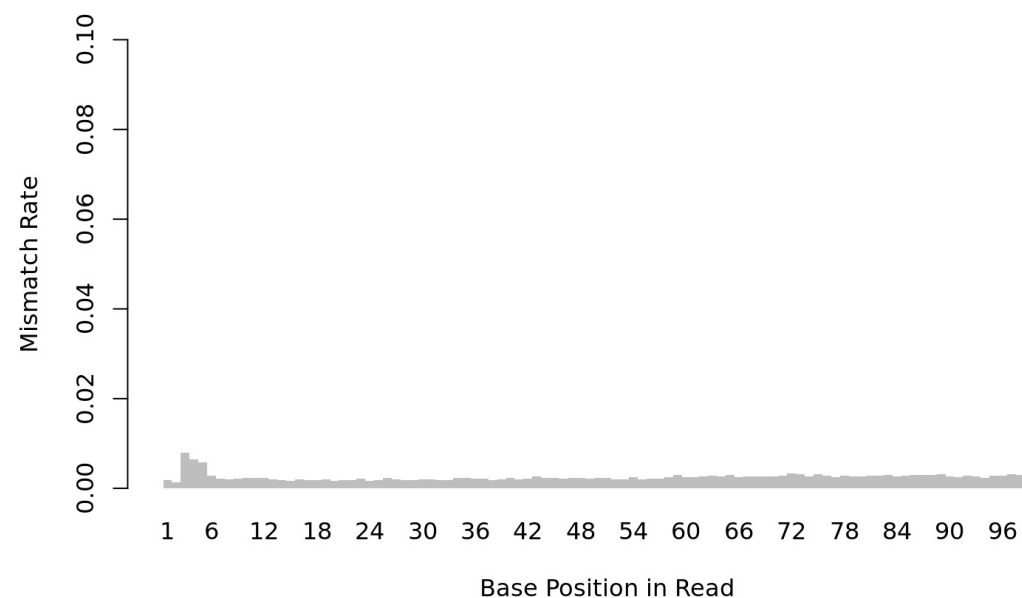
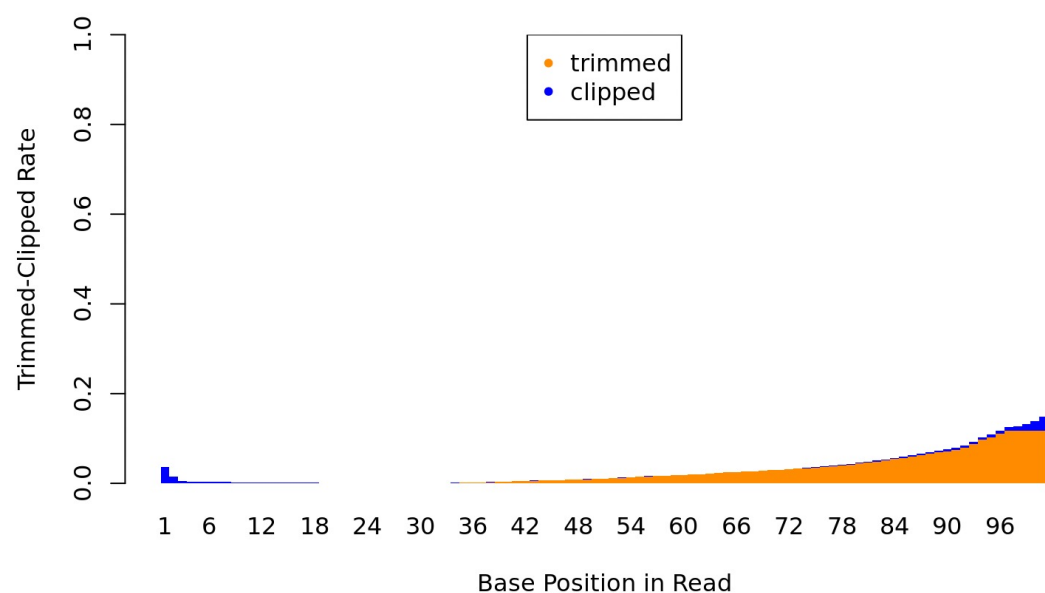


Mapping QC: Junction saturation



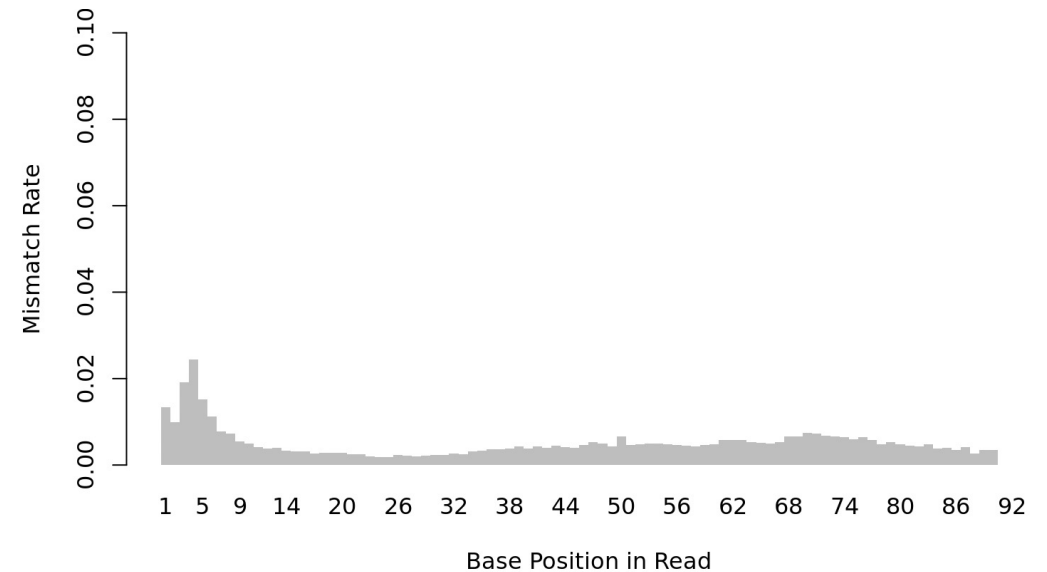
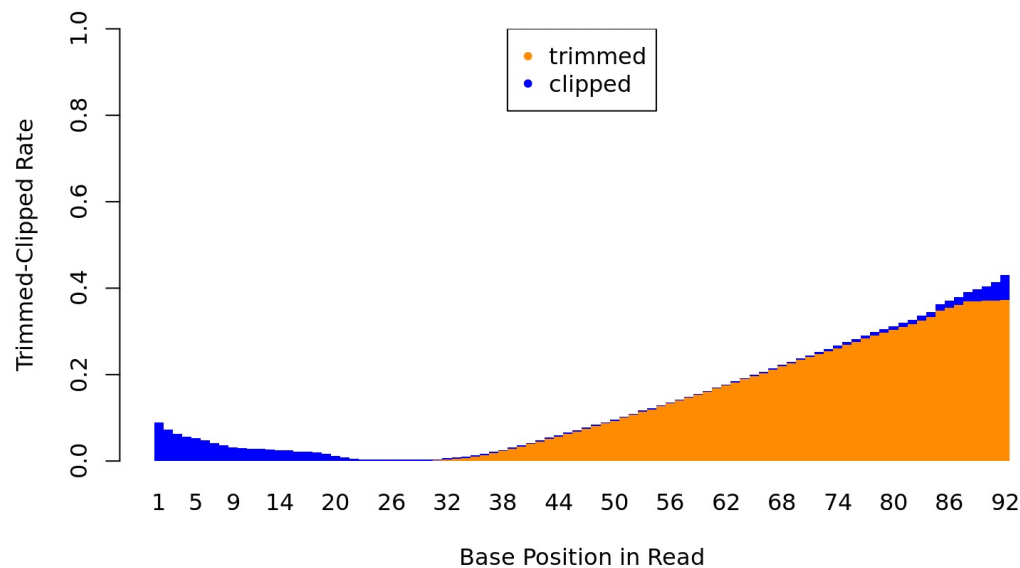
- Depth needed for alternative splicing analysis
- All annotated splice junctions are detected - a saturated RNA-seq dataset

Statistics on Trimming, Soft Clipping and Mismatches



- Mismatches depend on position in the read
- If bases 1 or 2 do not match, the STAR aligner soft-clips them

Statistics on Trimming, Soft Clipping and Mismatches



- Example from a low quality sample (low input <1ng, slight degradation)
- Short inserts, increased mismatch rate

RNA-seq mapping QC tools

- RNA-SeQC
 - <https://github.com/getzlab/rnaseqc>
- RSeQC
 - <http://rseqc.sourceforge.net/>

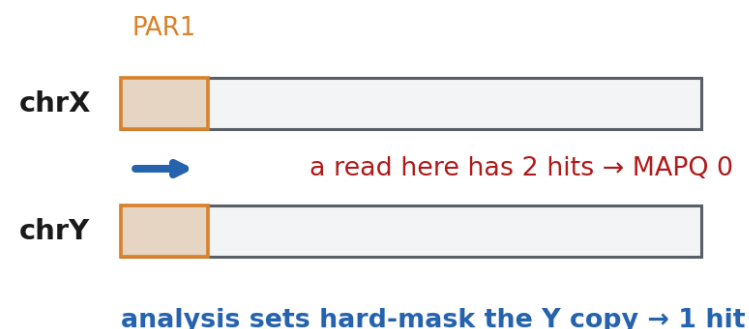
Where mappers have issues

- paralog genes that exist on X and Y chromosomes
- MHC genes (part of the adaptive immune response)
 - poly-morphic: several alleles (versions) exist on that gene's locus
- both are influenced by the reference

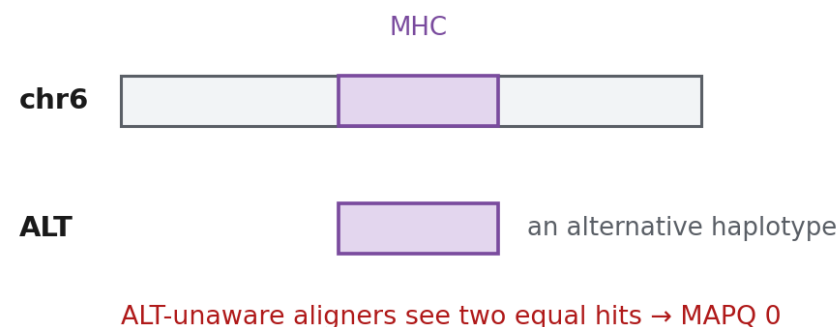
Choosing the Reference Is Part of the Analysis

- A read can only be placed on sequence that is in the index: the reference defines the possible answers
- GRCh38 comes in several versions — use an **analysis set**:
 - primary assembly only: ALT contigs give MAPQ 0 unless the aligner is ALT-aware
 - chrY PARs hard-masked, so PAR reads are not split between X and Y
 - decoys (hs38d1, EBV) absorb reads that would otherwise land elsewhere
- T2T-CHM13 (2022): the first complete human genome, adding ≈ 200 Mb of mostly repetitive sequence
- Annotation must match: same assembly, same chromosome names (chr1 vs 1); record the GENCODE / Ensembl release
- reference and annotation versions are parameters of the analysis and you need to report them

a) PARs are identical on X and Y

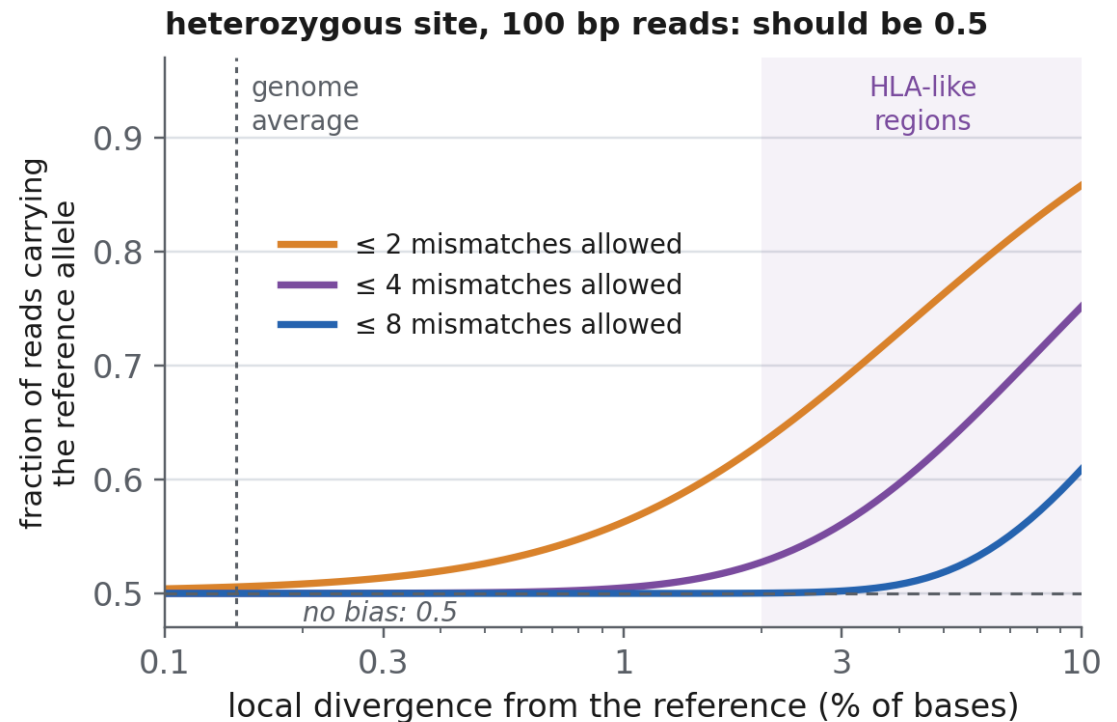


b) ALT contigs duplicate variable regions



Reference Bias Is a Selection Effect

- At a heterozygous site, a read with the alternative allele carries one more mismatch than the same read with the reference allele
- Non-reference allele may fail to align correctly → the reference allele is over-counted



Model: 100-bp read, 0.1% sequencing errors, local divergence d at the other 99 bases; a read is kept if it has $\leq m$ mismatches.

→ allele-specific expression, eQTL, imprinting and heterozygous genotype calls are all biased — most where the genome is most polymorphic (MHC / HLA)